ЗВІТ

до лабораторної роботи з циклічних алгоритмів

Тема: обчислення добутку з використанням вкладеного та одного циклу

Виконав: Трохимчук Іван Васильович

Група: ІО-62

Викладач: Русінов Володимир В.

2026

1. Мета роботи

Ознайомитися з побудовою циклічних алгоритмів, розглянути вкладений цикл та порівняти його з алгоритмом, у якому проміжна сума накопичується в одному циклі.

2. Алгоритм обчислення

Обидві програми зчитують ціле число n, початково задають P = 1 і для кожного i від 1 до n обчислюють множник $S_i / (i \cdot (i + 1))$, де S_i є сумою значень sin(k) + 1 для k від 1 до i. Функція sin у C приймає аргумент у радіанах.

Формули алгоритму:

```
S_i = sum(k = 1..i) (sin(k) + 1)
P = product(i = 1..n) [S_i / (i * (i + 1))]
```

У першій програмі S_i перераховується з нуля для кожного i за допомогою внутрішнього циклу j. У другій програмі змінна sum накопичується поступово: на кроці i додається тільки sin(i) + 1. Обидва підходи формують однакові значення S_i та мають давати однаковий результат для того самого n.

3. Лістинг програми № 1

```c
#include <stdio.h>
#include <math.h>

int main()
{
    int n;
    double P = 1.0;
    printf("Enter n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        double sum = 0.0f;
        for (int j = 1; j <= i; j++)
        {
            sum += sin(j) + 1;
        }
        P *= sum / (i * (i + 1));
    }
    printf("%.7f\n", P);

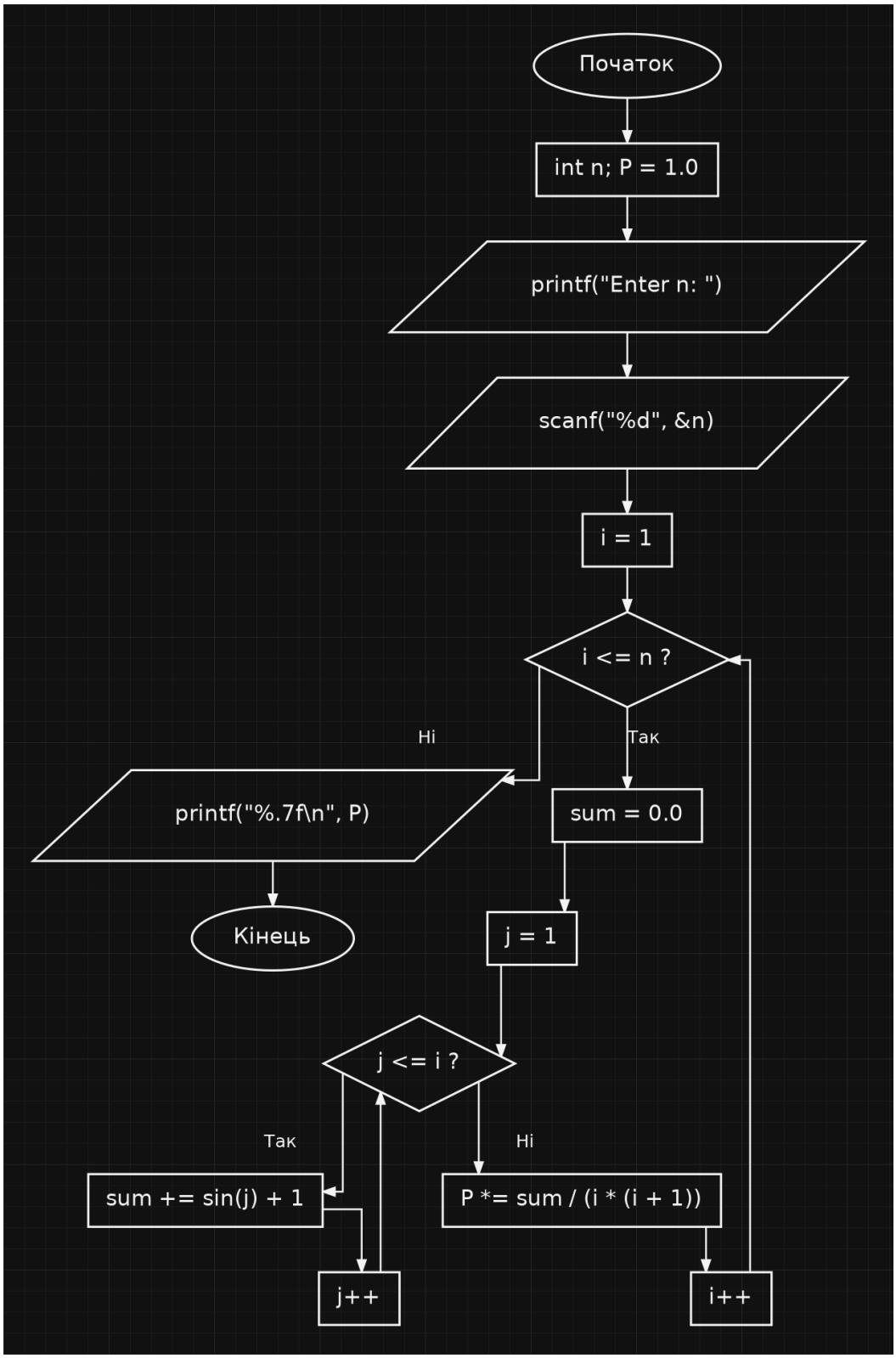


    return 0;
}
```

Особливість: для кожного i змінна sum спочатку обнуляється, після чого внутрішній цикл підсумовує sin(j) + 1 від j = 1 до j = i. Це пряме обчислення кожної часткової суми.

4. Блок-схема програми № 1

Схема показує зовнішній цикл за i та внутрішній цикл за j. Після завершення внутрішнього циклу обчислюється черговий множник добутку P.



5. Лістинг програми № 2

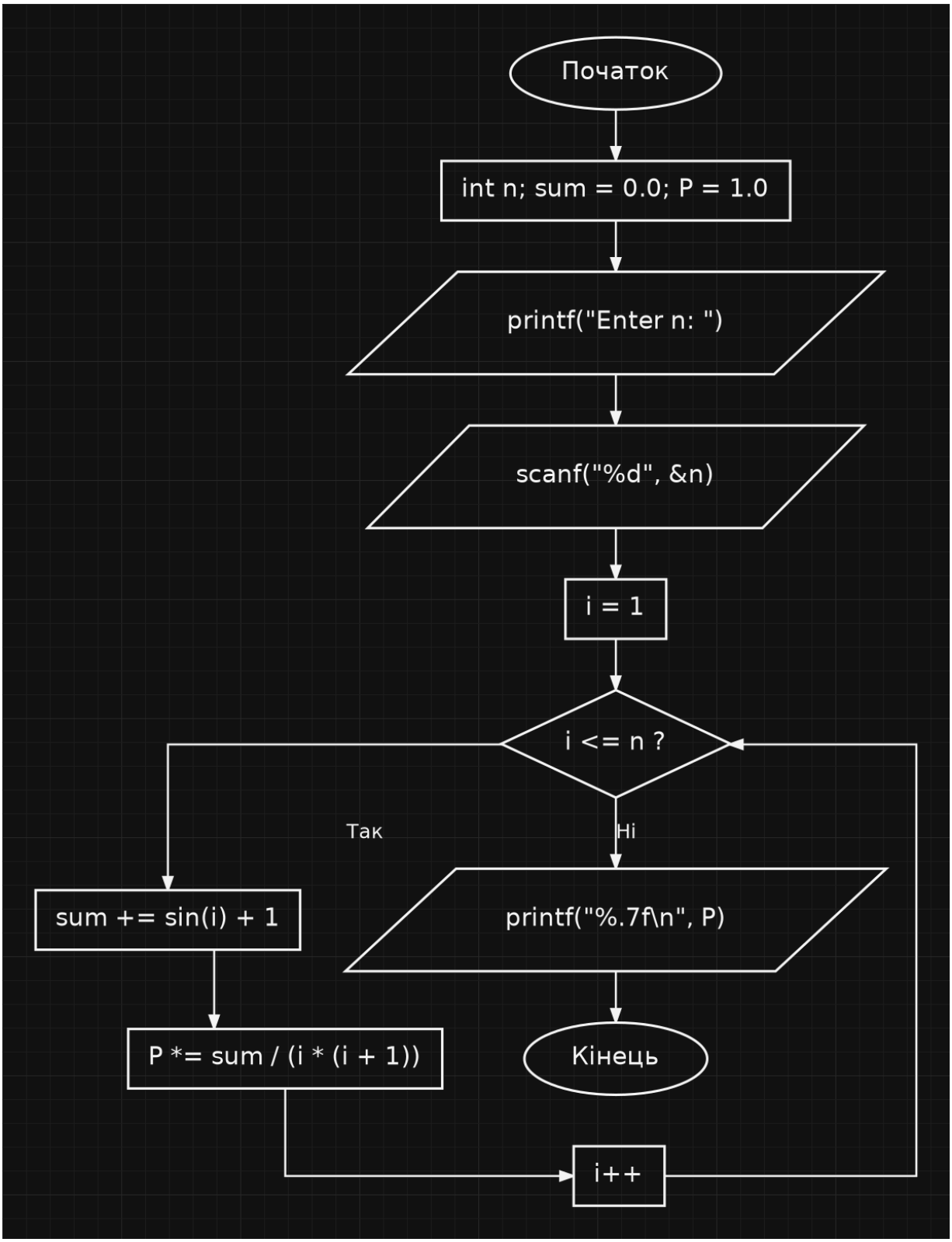
```
#include <stdio.h>
#include <math.h>

int main()
{
    int n;
    double sum = 0.0;
    double P = 1.0;
    printf("Enter n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        sum += sin(i) + 1;
        P *= sum / (i * (i + 1));
    }
    printf("%.7f\n", P);
    return 0;
}
```

Особливість: змінна sum оголошується перед зовнішнім циклом і не скидається на кожній ітерації. Тому вона зберігає суму всіх доданків до поточного і включно. Внутрішній цикл не потрібний.

6. Блок-схема програми № 2

Після перевірки умови i <= n програма додає черговий доданок, оновлює добуток P, збільшує i та повертається до перевірки циклу.



7. Перевірка результатів

Нижче наведено контрольні результати для кількох значень n. За n ≤ 0 цикл не виконується, тому P залишається рівним 1.0000000. Обидві програми мають вивести однакові значення з точністю до семи знаків після коми.

n	Програма № 1	Програма № 2
0	1.0000000	1.0000000
1	0.9207355	0.9207355
2	0.5755776	0.5755776
3	0.2346384	0.2346384
5	0.0103945	0.0103945

Порівняння алгоритмів

Обидві програми обчислюють однаковий добуток. Програма № 1 повторно рахує кожну часткову суму й виконує приблизно $n(n + 1)/2$ додавань, тому її складність за кількістю операцій становить $O(n^2)$. Програма № 2 накопичує суму за один прохід і має складність $O(n)$, тож вона ефективніша для великих n.

Висновок

Було розглянуто дві реалізації одного обчислювального алгоритму. Вкладений цикл у першій програмі та накопичення суми в другій дають однакові результати, але друга реалізація потребує менше операцій. Для однакового результату програмі достатньо одного циклу, оскільки попередня сума вже доступна на кожному наступному кроці.